

Mentor AI — Emotion Classification Model

Training, evaluation and integration report

Final Year Project · Generated 14 September 2026 from the project's saved experiment artefacts
Deployed model version: v2-fs2-linear_svm_calibrated-20260914

1. Summary

Mentor AI is a mentoring application (React PWA + Node.js/Express + PostgreSQL) whose replies are produced by a large language model. This project adds a **supervised emotion classifier** that reads the user's message or check-in and returns an emotion label with a calibrated confidence. The label is passed to the LLM as structured context so the mentor can adapt its tone. It is emotion classification of text, **not a medical or psychological diagnosis**, and the LLM remains responsible for the final response.

Headline results (all measured, none estimated).

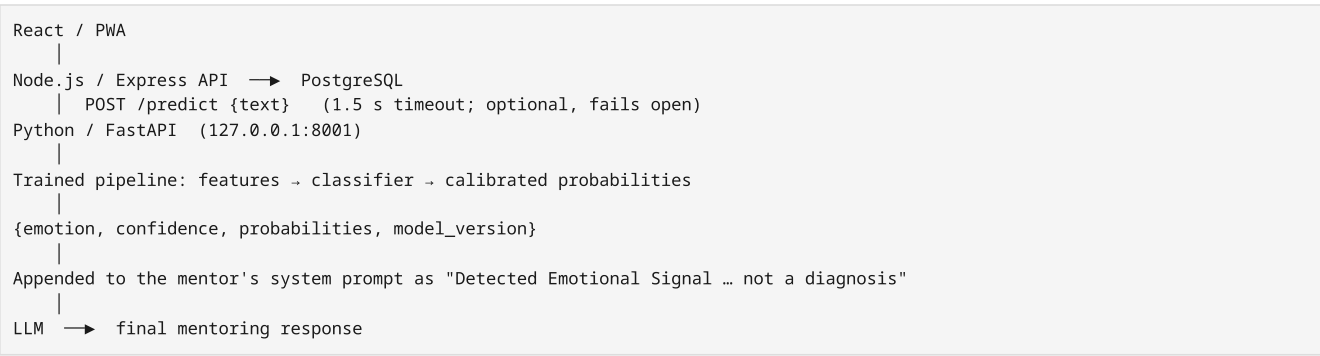
Deployed model: TF-IDF (word + character n-grams) + linear SVM with isotonic calibration.

GoEmotions test set (used once per model version): accuracy **0.779**, macro F1 **0.686**, weighted F1 0.772, expected calibration error 0.026.

Mentor-domain check-in sentences (154 items, draft labels): accuracy **0.383**, macro F1 **0.389** — this is the score that reflects real use, and it shows a large domain gap between Reddit training data and journal-style text.

Seven emotion classes are predicted: joy, sadness, fear_anxiety, anger_frustration, guilt, motivation, neutral. The class set was derived from the guide's suggested taxonomy, then adjusted on measured evidence (section 4.3).

2. Role in the Mentor AI architecture



Design rules: the classifier is *advisory*. If the service is down, slow, or its confidence is below a threshold (EMOTION_MIN_CONFIDENCE, recommended 0.6), no emotion context is sent and the chat behaves exactly as before. The label is never shown to the user as a fact; the prompt instructs the LLM to use it only to adapt tone and support.

3. Environment and reproducibility

Item	Value
Hardware	Lenovo LOQ laptop: 12-core CPU, 15 GiB RAM, NVIDIA GeForce RTX 3050 6 GB (used only for the optional transformer)
Operating system	Fedora Linux 44 (kernel 7.1.13-200.fc44.x86_64)
Python / packages	Python 3.12.14, scikit-learn 1.9.1, NumPy 2.5.3, pandas 3.0.5
Environment manager	uv with a lock file (uv.lock) and an exported requirements.txt
Random seed	42 everywhere (splits are the official GoEmotions splits; CV folds, classifiers and the transformer are seeded)
Frozen data	SHA-256 of each processed split is stored in data/processed/manifest.json; every script verifies it before loading. Rebuilding the data twice produced identical hashes.

The full pipeline is one command, `make all` (= data → train → tune → evaluate → test), following the guide's exact training order. Notebooks `01_dataset_exploration` and `02_model_experiments` read their figures from the same files.

4. Dataset

4.1 Source

GoEmotions (Demszky et al., ACL 2020; google-research-datasets/go_emotions (simplified), licence CC BY 4.0): 58k English Reddit comments annotated with 27 emotions plus neutral, multi-label, with official train/validation/test splits of 43,410 / 5,426 / 5,427 examples. It was chosen for its fine-grained labels and permissive licence. Its weaknesses — Reddit register, crowd-sourced label noise, and a different domain from journaling — are analysed in section 7.

4.2 Label mapping (version v2)

The guide suggested Joy, Sadness, Fear/Anxiety, Anger, Guilt, Frustration, Motivation and Neutral. GoEmotions labels were mapped onto these classes; labels that describe attitudes towards others (admiration, approval, caring), cognitive states (confusion, curiosity, realization) or are valence-ambiguous (surprise, disapproval) were dropped: admiration, approval, caring, confusion, curiosity, disapproval, realization, surprise.

Mentor AI class	GoEmotions source labels
joy	joy, amusement, excitement, gratitude, relief, love
sadness	sadness, grief
fear_anxiety	fear, nervousness
anger_frustration	anger, disgust, annoyance, disappointment
guilt	remorse, embarrassment
motivation	optimism, desire, pride
neutral	neutral

An example is kept **only if all of its labels map to the same Mentor class**; mixed-emotion comments are removed rather than force-labelled. This is a deliberate trade-off: cleaner single-label training data at the cost of about 37% of the raw rows.

4.3 Evidence-based change: merging anger and frustration

The first mapping (v1) kept anger and frustration separate. Measured on the validation set:

v1 class	train support	E1 LogReg F1	E2 LinearSVM F1
fear_anxiety	536	0.6087	0.6429
guilt	559	0.6308	0.6783
anger	1551	0.4869	0.4709
frustration	2232	0.2934	0.2650
macro F1 (8 classes)		0.5951	0.6130

fear_anxiety and guilt were below the 800-example review threshold but performed in line with the other minority classes, so they were kept. frustration had ample data yet F1 of only ~0.28, being confused mostly with anger and neutral. The two were merged into anger_frustration (v2). Validation macro F1 rose from 0.613 to 0.672 for the same SVM. The decision was made on validation data only, before the test set was touched.

4.4 Processed splits

class	train	validation	test
joy	6743	861	814
sadness	868	91	107
fear_anxiety	536	72	80
anger_frustration	4119	497	551
guilt	559	61	67
motivation	1341	186	176
neutral	12781	1591	1605
total	26947	3359	3400

step	train	validation	test
raw_rows	43410	5426	5427
kept_after_mapping	27134	3368	3405
removed_only_unmapped_labels	11182	1426	1431
removed_partially_unmapped_labels	3539	437	409
removed_conflicting_mentor_classes	1555	195	182
removed_empty_text	0	0	0
removed_exact_duplicates	108	2	5
removed_same_text_different_labels	38	0	0
removed_overlap_with_validation_or_test	41	0	0
removed_overlap_with_test	0	7	0

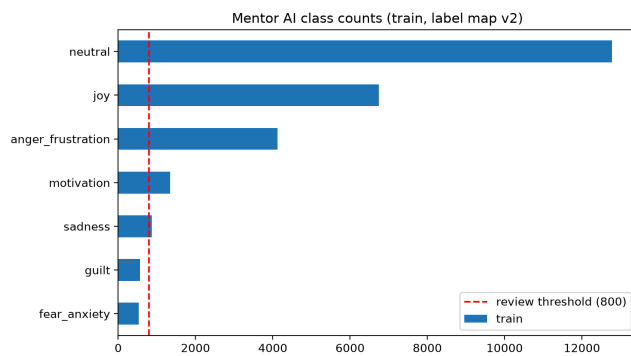


Figure 1. Training examples per Mentor AI class (v2).

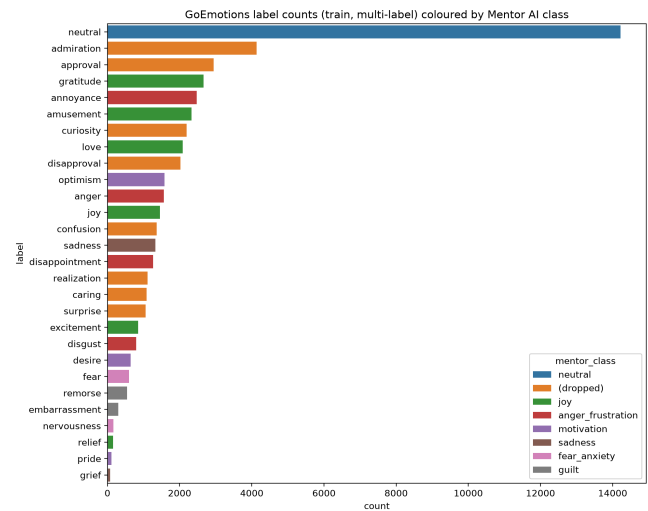


Figure 2. Raw GoEmotions label counts coloured by Mentor AI class.

5. Pre-processing and leakage prevention

Pre-processing is deliberately conservative, following the guide: whitespace is normalised and text trimmed; empty texts, exact duplicates and texts that appear with conflicting labels are removed; texts that also occur in a later split are removed from the earlier one. Negations, punctuation, emojis, casing and stop words are all kept, and no stemming is applied, because these carry emotional signal.

Leakage is prevented structurally: every model is a scikit-learn Pipeline whose vectorisers are fitted inside `fit()`, so vocabulary and IDF statistics come from training text only; cross-validation happens inside that pipeline; and an automated test asserts that an unseen word never enters the vocabulary. The test set is loaded only by `src/evaluate.py`, which refuses to run a second time without an explicit, logged reason.

6. Methodology and experiments

6.1 Feature sets

fs1 — the guide's configuration: word uni- and bi-gram TF-IDF (lower-cased, sublinear TF, `max_df` 0.95). Review of the trained model showed that scikit-learn's default token pattern silently discards emojis and punctuation, contradicting the "keep emojis" rule. **fs2** fixes this with a token pattern that keeps single characters and symbols, and adds character n-grams (2–5, within word boundaries) to capture informal spelling and typos. fs2 improved every model (table below).

6.2 Models

E1 Logistic Regression (`class_weight="balanced"`, `max_iter=2000`) — fast, interpretable, native probabilities. **E2** Linear SVM (`LinearSVC`, `balanced`) — typically strong on sparse text but gives no probabilities. **E3** the best of both after a controlled grid search. **E4** DistilBERT (optional transformer phase).

6.3 Hyper-parameter search (train only)

3-fold stratified cross-validation on the training split with macro F1 as the criterion, over word n-gram range, minimum document frequency and C for both classifiers (fs1: 36 candidates; fs2: 12 candidates, because character n-grams make each fit ~4× slower). The best configuration was then frozen and checked once on validation. Top fs2 candidates:

rank CV F1	mean CV F1	std CV F1	mean_fit_time	clf	clf_C	ngram_range	min_df
1	0.6629	0.0076	4.1336	LinearSVC	0.2500	(1, 2)	1
2	0.6613	0.0060	3.8517	LinearSVC	0.2500	(1, 2)	2
3	0.6577	0.0044	4.4659	LinearSVC	0.5000	(1, 2)	1
4	0.6521	0.0063	4.2132	LinearSVC	0.5000	(1, 2)	2
5	0.6472	0.0045	5.0360	LinearSVC	1.0000	(1, 2)	1
6	0.6457	0.0066	35.1825	LogisticRegression	1.0000	(1, 2)	1
7	0.6452	0.0077	26.0618	LogisticRegression	1.0000	(1, 2)	2
8	0.6407	0.0049	4.6956	LinearSVC	1.0000	(1, 2)	2

6.4 Calibration

The API must return a real probability, and the Node side thresholds on it. An SVM decision score is not a probability, so the deployed SVM is wrapped in `CalibratedClassifierCV`. Four options were compared on validation: sigmoid (Platt) vs isotonic, and an ensemble of three fold-models vs a single full-data model. Sigmoid/ensemble — the common default — lost ~2 points of macro F1 because it shifts decisions towards the majority class; isotonic on a single full-data model kept the SVM's macro F1 (0.711 vs 0.715) and had the lowest expected calibration error (0.027). It is the deployed choice.

6.6 Experiment log (validation unless marked test)

id	features	model	eval_split	accuracy	macro_precision	macro_recall	macro_f1	weighted_f1	cv_macro_f1	purpose
E1	fs1	logreg	validation	0.7312	0.6198	0.6871	0.6483	0.7360		Baseline
E2	fs1	linear_svm	validation	0.7502	0.6833	0.6635	0.6722	0.7487		Comparison
E3	fs1	linear_svm	validation	0.7681	0.7059	0.6886	0.6962	0.7665	0.6356	Best candidate (tuned)
E3-cal	fs1	linear_svm_calibrated	validation	0.7705	0.7648	0.6418	0.6920	0.7636	0.6356	Best candidate with calibrated probabilities
E3-final	fs1	linear_svm_calibrated	test	0.7668	0.7481	0.6238	0.6720	0.7562	0.6356	Final model (single test evaluation)
E1	fs2	logreg	validation	0.7499	0.6510	0.7071	0.6745	0.7550		Baseline
E2	fs2	linear_svm	validation	0.7672	0.7195	0.6788	0.6972	0.7655		Comparison
E3	fs2	linear_svm	validation	0.7800	0.7154	0.7160	0.7149	0.7783	0.6629	Best candidate (tuned)
E3-cal	fs2	linear_svm_calibrated	validation	0.7895	0.7803	0.6395	0.6954	0.7816	0.6629	Best candidate with calibrated probabilities
E3-final	fs2	linear_svm_calibrated	test	0.7794	0.7357	0.6542	0.6862	0.7718	0.6629	Final model (single test evaluation)
E3-iso	fs2	linear_svm_calibrated	validation	0.7854	0.7545	0.6779	0.7110	0.7797	0.6629	Deployed calibration (isotonic, chosen on validation)

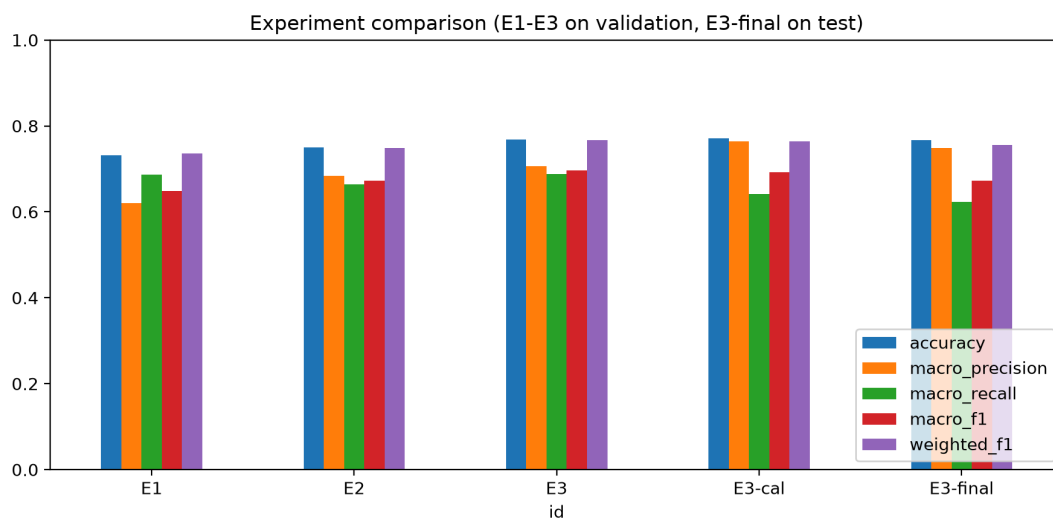


Figure 3. Headline metrics per experiment.

6.7 Per-class F1 on validation

class	E1 / fs2	E1 / fs1	E2 / fs2	E2 / fs1	E3-cal / fs2	E3-cal / fs1	E3-iso / fs2	E3 / fs2	E3 / fs1
joy	0.8627	0.8478	0.8629	0.8522	0.8766	0.8583	0.8712	0.8714	0.8672
sadness	0.5286	0.5244	0.6270	0.5231	0.5695	0.5789	0.6265	0.6114	0.6010
fear_anxiety	0.6447	0.5987	0.6567	0.6277	0.6613	0.6562	0.7015	0.6944	0.6522
anger_frustration	0.5829	0.5382	0.5585	0.5361	0.5820	0.5529	0.5765	0.5844	0.5639
guilt	0.6774	0.6212	0.6903	0.7009	0.6667	0.7037	0.6957	0.7258	0.6957
motivation	0.6402	0.6359	0.6822	0.6780	0.6861	0.6897	0.6852	0.7042	0.6923
neutral	0.7848	0.7717	0.8029	0.7876	0.8257	0.8046	0.8202	0.8126	0.8014

7. Results

7.1 Final test-set results (one evaluation per model version)

Each row is a distinct model version evaluated once on the untouched test set after its configuration was frozen on validation. Versions were *not* selected by their test score.

model version	accuracy	macro P	macro R	macro F1	weighted F1	ECE
v2-linear_svm_calibrated-20260914	0.7668	0.7481	0.6238	0.6720	0.7562	
v2-fs2-linear_svm_calibrated-20260914	0.7794	0.7357	0.6542	0.6862	0.7718	0.0262

7.2 Deployed model — per-class test results

class	precision	recall	F1	support
joy	0.8530	0.8980	0.8749	814
sadness	0.6974	0.4953	0.5792	107
fear_anxiety	0.6585	0.6750	0.6667	80
anger_frustration	0.6762	0.5154	0.5850	551
guilt	0.6724	0.5821	0.6240	67
motivation	0.8136	0.5455	0.6531	176
neutral	0.7786	0.8679	0.8209	1605

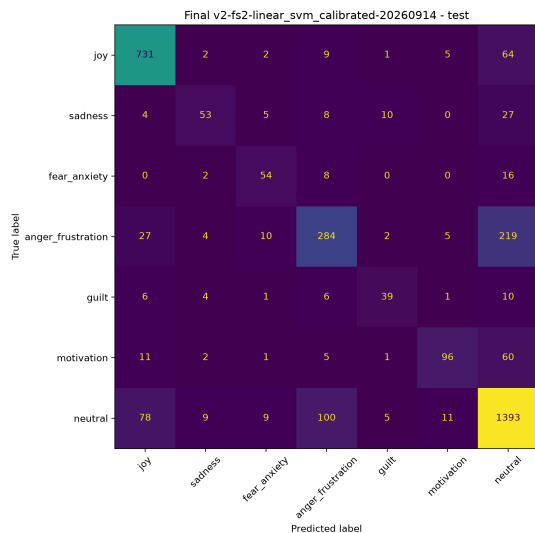


Figure 4. Test confusion matrix (counts), v2-fs2-linear_svm_calibrated-20260914.

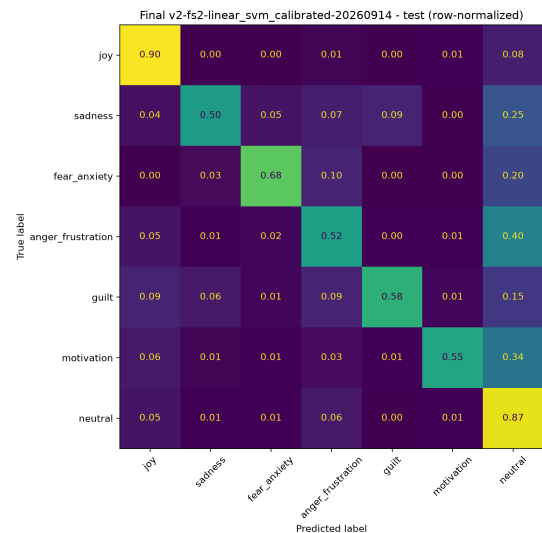


Figure 5. Row-normalised: share of each true class predicted as each label.

Reading the matrix: joy and neutral are recognised well; the minority classes are under-detected, with most misses going to neutral. Because the app sends context only for confident non-neutral predictions, a miss usually means "no context" (harmless) rather than a wrong emotion.

7.3 Confidence calibration and the Node threshold

Expected calibration error on test: **0.026** (0 = perfect). The table shows, for each EMOTION_MIN_CONFIDENCE, how often a non-neutral emotion would be injected into the prompt and how often it would be right. A wrong emotion in the prompt is worse than none, so 0.6 is

recommended.

threshold	coverage	accuracy of predictions above threshold	share of messages that get context	precision of that context
0.000	1.000	0.779	0.474	0.780
0.400	0.979	0.789	0.462	0.794
0.500	0.908	0.814	0.421	0.834
0.600	0.784	0.851	0.349	0.877
0.700	0.657	0.876	0.290	0.909

7.4 Mentor-domain evaluation set

GoEmotions scores measure Reddit comments. To measure what matters for the app, a set of 154 check-in / journal style sentences (22 per class, including negation, emoji and very short items) was written and scored. It is used for evaluation only, never for training or tuning.

Status: the labels of this set are a draft written by the developer's assistant and have not yet been reviewed by the project owner. Treat these numbers as provisional until data/mentor_eval/README.md records a reviewer.

model version	accuracy	macro F1	acc (emoji)	acc (negation)	acc (plain)	acc (short)	wrong emotion sent @0.6
v2-fs2-linear_svm_calibrated-20260914	0.422	0.431	0.125	0.500	0.419	0.714	9
v2-linear_svm_calibrated-20260914	0.383	0.389	0.125	0.500	0.372	0.714	5

class	v2-fs2-linear_svm_calibrated-20260914	v2-linear_svm_calibrated-20260914
joy	0.5294	0.6154
sadness	0.2857	0.2759
fear_anxiety	0.5161	0.3571
anger_frustration	0.5116	0.4324
guilt	0.5333	0.4286
motivation	0.2667	0.2759
neutral	0.3750	0.3390

[missing figure: domain_eval_comparison.png]

Typical errors of the latest model on this set (first 20):

id	text	expected	predicted	confidence
J02	Finally submitted the thesis draft. I can't stop smiling.	joy	neutral	0.60
J04	we did it!!! 🎉🎉 first 10k done	joy	neutral	0.67
J05	Got the internship offer this morning 😊	joy	neutral	0.67
J07	My code passed all the tests on the first try, what a feeling.	joy	neutral	0.57
J08	Spent the afternoon in the park with friends, exactly what I needed.	joy	neutral	0.81
J12	Ten days sugar-free and I feel lighter and happier.	joy	neutral	0.64
J13	My mom called just to say she's proud of me 😊	joy	motivation	0.61
J16	Honestly grateful for this app, the check-ins are keeping me sane.	joy	neutral	0.44
J18	Went for a walk at sunrise. Everything felt okay for a moment.	joy	neutral	0.65
J21	Reconnected with an old friend, we talked for three hours.	joy	neutral	0.76
S01	Lost my grandmother last week. Hard to focus on anything.	sadness	anger_frustration	0.37
S02	I don't know if I can keep this up, everything feels pointless lately.	sadness	neutral	0.59
S03	Nobody remembered my birthday.	sadness	neutral	0.51
S04	Came home to an empty flat again. Just feeling low tonight.	sadness	neutral	0.62
S05	My best friend is moving abroad next month and I'm gutted.	sadness	neutral	0.63
S06	Cried in the car after work. Not even sure why.	sadness	neutral	0.68
S07	Three rejections this week. Starting to feel like I'm not good enough.	sadness	anger_frustration	0.54
S10	My dog is sick and the vet doesn't sound hopeful.	sadness	neutral	0.38
S11	Another weekend alone. It's getting to me.	sadness	anger_frustration	0.52
S12	Didn't get into the program. I really wanted this one.	sadness	motivation	0.47

The pattern is consistent: sentences that express an emotion through a *situation* ("stomach in knots", "skipped my workout again") rather than an explicit emotion word are classified as neutral, and negation ("I'm not happy") is not understood by a bag-of-words model. These are limits of the training data's domain and of TF-IDF, not bugs — and they are exactly what the transformer phase and domain-specific data address.

8. Deployment and integration

Model service (app/main.py, FastAPI/uvicorn on 127.0.0.1:8001): loads the serialised pipeline once at start-up; POST /predict {"text"} returns {emotion, confidence, probabilities, model_version}; empty or over-long text is rejected with 422; GET /health reports the model version and classes; interactive docs at /docs. A systemd user unit is provided. Latency of the classical model is ~2 ms per message on CPU.

Node.js integration (Mentor AI API, TypeScript): a new EmotionClassifierService calls the service with a 1.5 s timeout, validates the JSON with Zod and returns null on any failure or low confidence. ChatAgentService runs the classification *concurrently* with building the system prompt, so no latency is added, then appends a short "Detected Emotional Signal" section that explicitly says the signal may be wrong and is not a diagnosis. The feature is behind EMOTION_CLASSIFIER_ENABLED (off by default), so production is unaffected until the service is deployed.

9. Limitations, ethics and future work

- **Domain shift** — training data is Reddit comments; users write journal entries and check-ins. This is the dominant source of error and is quantified by the Mentor-domain set.
- **Label noise and approximation** — crowd-sourced labels; guilt (remorse + embarrassment) and motivation (optimism + desire + pride) are approximations of the intended classes.
- **Class imbalance and small classes** — neutral is 47% of training data; fear_anxiety and guilt have fewer than 600 training and under 100 test examples, so their scores have wide uncertainty.
- **Bag-of-words** — word order, negation scope and sarcasm are not modelled by TF-IDF.
- **Ethics** — the output is a coarse, uncertain signal about text, never a statement about the person; it is not shown to the user as a fact, is thresholded on confidence, fails open, and must not be used for any clinical purpose.

Future work: (1) collect a consented, anonymised set of real Mentor AI check-ins and use it both as the acceptance test and, with a separate held-out part, as domain training data; (2) fine-tune a transformer on the combined data; (3) extend the signal to voice sessions and store per-message emotions to give mentors trend insight over time; (4) revisit the taxonomy once real-domain data shows which distinctions users actually make.

Appendix A. Reproduction commands

```
sudo dnf install -y uv python3.12
cd ~/Coding/python && uv sync                # environment from uv.lock
make data      # download GoEmotions, map labels, clean, freeze splits (hashes in manifest.json)
make train     # E1 Logistic Regression, E2 Linear SVM -> validation metrics
make tune      # E3 grid search (3-fold CV on train) -> models/frozen_config.json
make evaluate   # single test-set evaluation, serialise models/emotion_classifier.joblib
make test       # pytest: data hygiene, leakage guard, API contract
make domain-eval # score on data/mentor_eval/mentor_eval.csv
uv sync --extra transformer && make evaluate-transformer # optional E4
make serve      # FastAPI on 127.0.0.1:8001
uv run python -m src.report # this document
```

Appendix B. Definition of done (guide §23)

- Dataset and label mapping documented — data/README.md, data/label_mapping.md
- Training reproducible — seeded, locked environment, hash-frozen splits (verified identical on rebuild)
- At least two models compared — E1, E2, E3 on identical splits
- Test set protected — loaded only by evaluate.py; one run per model version; guard and log for re-runs
- Metrics and confusion matrices saved — reports/metrics/, reports/figures/
- Final model serialised — models/emotion_classifier.joblib + model_card.json
- Local prediction API working — FastAPI, tests pass
- Node.js integration — implemented behind a feature flag; live end-to-end run pending
- Actual measured results and limitations — this report

Generated by src/report.py on 2026-09-14 19:51 UTC. Experiment rows: 11; test evaluations: 2.